Les interfaces

Le problème

On écrit un programme de gestion de garage. Il y a des voitures (`Car`) et des motos (`Bike`). Les deux peuvent démarrer, accélérer et freiner, mais chacune à sa manière : on démarre une voiture avec une clé ou un bouton, une moto avec le démarreur ou au kick.

Le garagiste, lui, veut faire un **essai routier** sur n'importe quel véhicule qu'on lui amène. Il ne veut pas écrire une méthode `TestCar(Car c)`, puis `TestBike(Bike b)`, puis `TestTruck(Truck t)` à chaque nouveau type de véhicule.

Ce qu'il lui faut, c'est une garantie : *« ce que tu m'amènes sait démarrer, accélérer et freiner »*. Peu importe comment.

C'est exactement ce qu'est une interface.

Une interface, c'est un contrat

Une interface est une **liste de capacités** (méthodes, propriétés) qu'une classe **s'engage à fournir**. Elle dit **quoi** faire, jamais **comment** le faire.

Une comparaison de la vie réelle : la **prise électrique**. La prise murale définit une forme et une tension. Un grille-pain, un chargeur de téléphone ou une lampe respectent tous cette « interface ». La prise ne sait pas ce qu'elle alimente, et les appareils ne savent pas d'où vient le courant. Chacun peut être changé sans toucher à l'autre.

En programmation orientée objet, c'est pareil :

- l'interface définit le contrat ;
- les classes qui **implémentent** l'interface remplissent le contrat ;
- le code qui **utilise** l'interface ne connaît que le contrat, pas la classe réelle.

Les interfaces en C#

Déclarer une interface

```
public interface IDrivable
{
    int Speed { get; }      // propriété en lecture
    void Start();
    void Accelerate(int amount);
    void Brake();
}
```

À remarquer :

- mot-clé `interface` au lieu de `class` ;

- par convention, le nom commence par un `I` **majuscule** (`IDrivable`, `IComparable`, `IDisposable`…) et décrit souvent une capacité (suffixe *-able*) ;
- les méthodes n'ont **pas de corps** : juste la signature suivie de `;` ;
- pas de modificateur d'accès sur les membres : ils sont **publics** d'office ;
- pas d'attributs (champs) : une interface ne stocke aucune donnée.

Implémenter l'interface

Une classe indique qu'elle implémente une interface avec `:`. Elle **doit** alors fournir une implémentation publique de **chaque** membre, sinon le code ne compile pas.

```
public class Car : IDrivable
{
    public int Speed { get; private set; }

    public void Start()
    {
        Console.WriteLine("Clé tournée, le moteur ronronne.");
    }

    public void Accelerate(int amount)
    {
        Speed += amount;
    }

    public void Brake()
    {
        Speed = Math.Max(0, Speed - 20);   // freinage sur 4 roues
    }
}
```

```
public class Bike : IDrivable
{
    public int Speed { get; private set; }

    public void Start()
    {
        Console.WriteLine("Un coup de kick, vroum !");
    }

    public void Accelerate(int amount)
    {
        Speed += amount * 2;               // plus légère, accélère plus vite
    }

    public void Brake()
    {
        Speed = Math.Max(0, Speed - 10);
```

```
    }
}
```

`Car` et `Bike` n'ont **aucun lien entre elles**. Elles partagent simplement le même contrat.

Utiliser l'interface comme type

C'est ici que l'interface devient utile. On peut déclarer une variable, un paramètre ou une liste **du type de l'interface**.

Faisons faire l'essai routier par des tests unitaires. Quel que soit le véhicule, on veut vérifier trois règles :

1. il va plus vite après avoir accéléré ;
2. il va moins vite après avoir freiné ;
3. sa vitesse n'est jamais négative, même si on freine à l'arrêt.

Dans les tests ci-dessous, la variable `vehicle` est déclarée de type `IDrivable`, et non `Car` ou `Bike` :

```
[TestClass]
public class DrivableTests
{
    [TestMethod]
    public void Test_that_car_goes_faster_after_accelerate()
    {
        // Arrange
        IDrivable vehicle = new Car();

        // Act
        vehicle.Accelerate(30);

        // Assert
        Assert.IsTrue(vehicle.Speed > 0);
    }

    [TestMethod]
    public void Test_that_car_goes_slower_after_brake()
    {
        // Arrange
        IDrivable vehicle = new Car();
        vehicle.Accelerate(50);
        int before = vehicle.Speed;

        // Act
        vehicle.Brake();

        // Assert
        Assert.IsTrue(vehicle.Speed < before);
    }

    [TestMethod]
    public void Test_that_car_speed_is_never_negative()
```

```
    {
        // Arrange
        IDrivable vehicle = new Car();

        // Act
        vehicle.Brake();

        // Assert
        Assert.AreEqual(0, vehicle.Speed);
    }

    [TestMethod]
    public void Test_that_bike_goes_faster_after_accelerate()
    {
        // Arrange
        IDrivable vehicle = new Bike();

        // Act
        vehicle.Accelerate(30);

        // Assert
        Assert.IsTrue(vehicle.Speed > 0);
    }

    [TestMethod]
    public void Test_that_bike_goes_slower_after_brake()
    {
        // Arrange
        IDrivable vehicle = new Bike();
        vehicle.Accelerate(50);
        int before = vehicle.Speed;

        // Act
        vehicle.Brake();

        // Assert
        Assert.IsTrue(vehicle.Speed < before);
    }

    [TestMethod]
    public void Test_that_bike_speed_is_never_negative()
    {
        // Arrange
        IDrivable vehicle = new Bike();

        // Act
        vehicle.Brake();

        // Assert
        Assert.AreEqual(0, vehicle.Speed);
    }
}
```

Les tests de la voiture et ceux de la moto sont identiques, à une ligne près : celle du `new`. Le reste du code ne connaît que le contrat `IDrivable`. Pourtant, quand on appelle `vehicle.Brake()`, c'est le `Brake()` de `Car` (−20) ou celui de `Bike` (−10) qui s'exécute, selon l'objet réellement créé : c'est du **polymorphisme**.

Le jour où l'on ajoute une classe `Truck : IDrivable`, il suffit de copier ces tests et de changer le `new`.

Attention : on ne peut **pas** instancier une interface. `new IDrivable()` ne compile pas : un contrat n'est pas un objet.

Implémenter plusieurs interfaces

Une classe peut implémenter **autant d'interfaces qu'elle veut**. C'est une des grandes forces des interfaces.

```
public interface ILockable
{
    bool IsLocked { get; }
    void Lock();
    void Unlock();
}

public interface ICargo
{
    int TrunkVolume { get; }   // en litres
}
```

```
public class Car : IDrivable, ILockable, ICargo
{
    // ... membres de IDrivable, ILockable et ICargo
}

public class Bike : IDrivable, ILockable
{
    // ... une moto n'a pas de coffre : pas de ICargo
}
```

Chaque classe choisit les contrats qui ont du sens pour elle. Un parking qui verrouille les véhicules la nuit pourra travailler avec une `List<ILockable>`, sans se soucier de la conduite ni du coffre.

À retenir

- Une interface est un **contrat** : elle dit **quoi**, la classe dit **comment**.
- En C# : mot-clé `interface`, nom commençant par `I`, membres sans corps et publics.
- Une classe qui implémente une interface **doit** en fournir tous les membres.
- On utilise l'interface comme **type** (paramètre, variable, liste) pour écrire du code qui fonctionne avec toutes les classes qui la respectent.
- Une classe peut implémenter **plusieurs** interfaces.
- On ne peut pas faire `new` sur une interface.